

Learning is a lifelong process of keeping your mind open to new ideas and experiences.

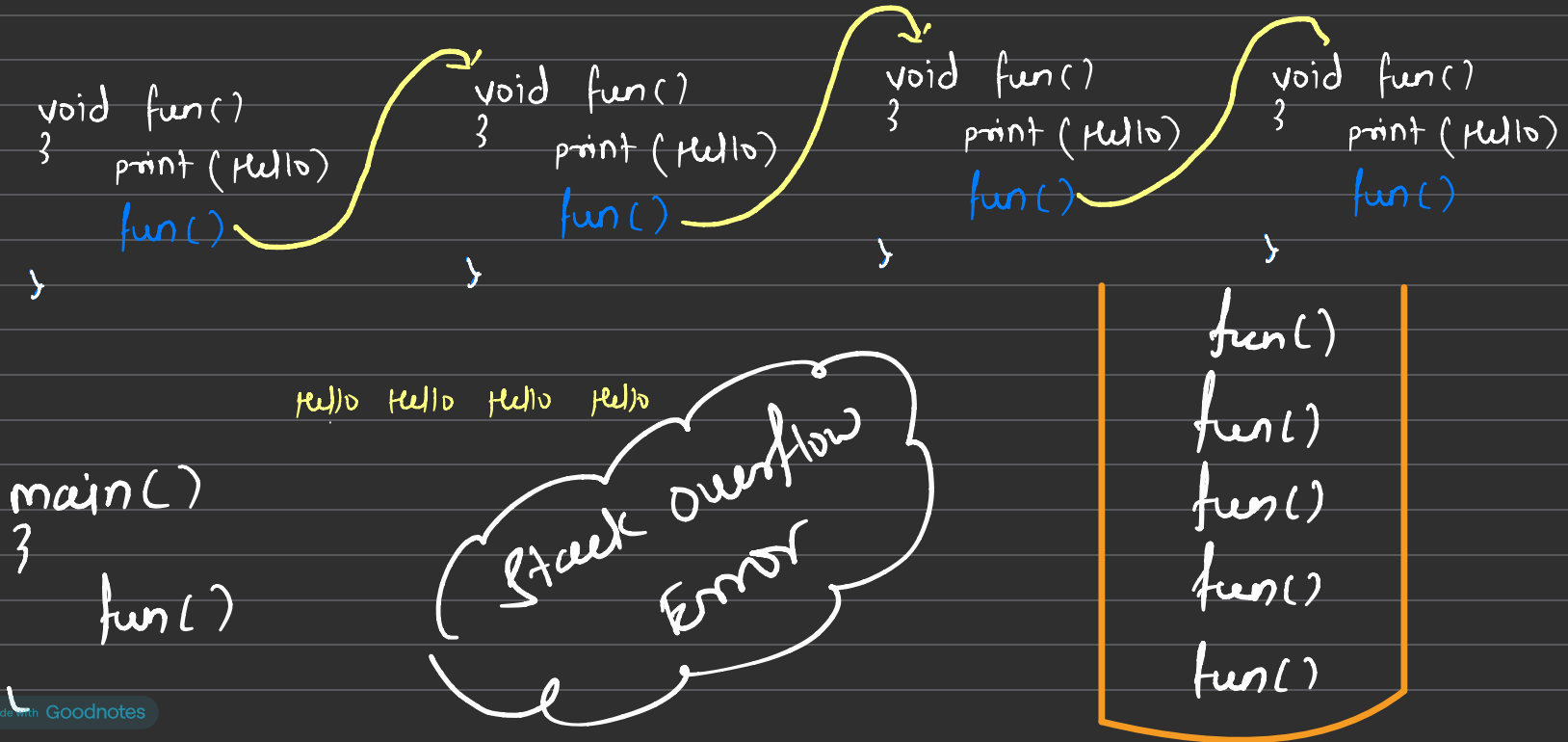
RECURSION & BACKTRACKING

- **RECURSIVE TREE**
- **DRY RUN**
- **FUNCTION CALL**



What is Recursion

Function calling itself until a specific condition is called as Recursion.



$cnt = 0$ ^{2/3}

```
void fun()  
{  
  if (cnt == 3)  
    return  
  cnt++  
  printf("Hello")  
  fun()  
}
```

```
void fun()  
{  
  if (cnt == 3)  
    return  
  cnt++  
  printf("Hello")  
  fun()  
}
```

```
void fun()  
{  
  if (cnt == 3)  
    return  
  cnt++  
  printf("Hello")  
  fun()  
}
```

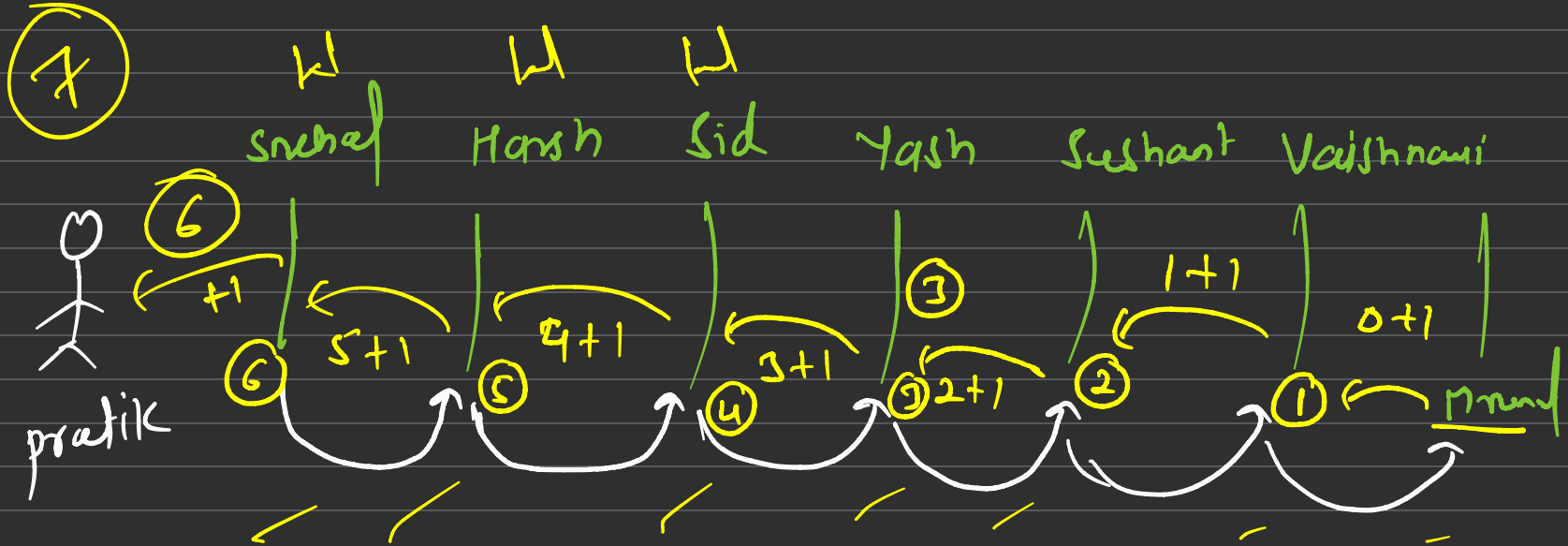
```
void fun()  
{  
  if (cnt == 3)  
    return  
  cnt++  
  printf("Hello")  
  fun()  
}
```

```
main()  
{  
  fun()  
}
```

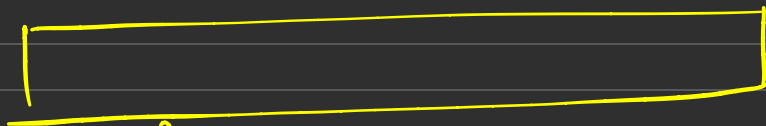
Base Case

Hello Hello Hello

Break large problem into smaller sub-problems

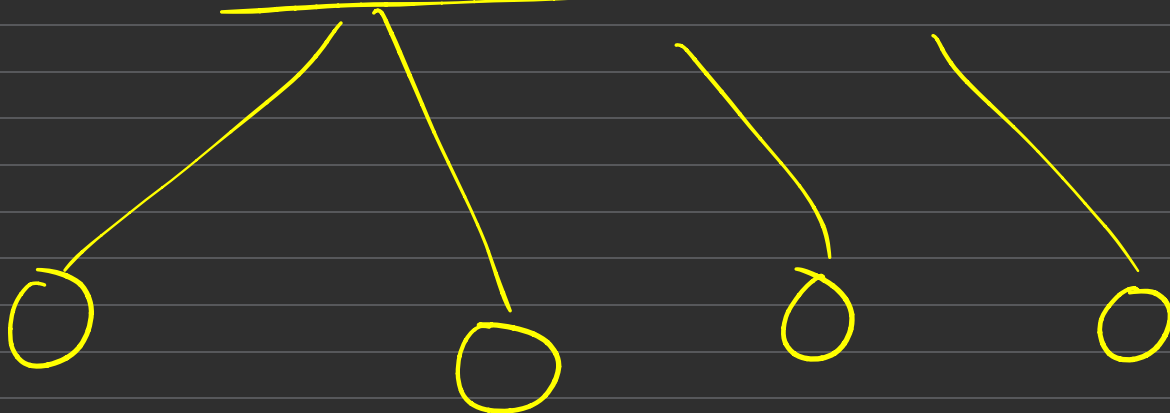


large

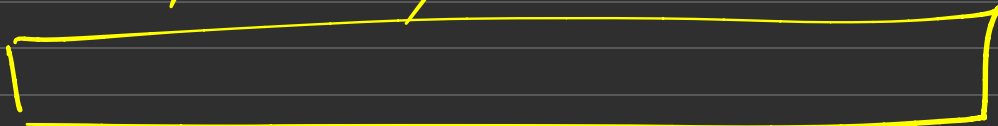


DnC

small



large



Activation Stack Record

```
void fun1()
{
    int x = 10;
    x++;
    print(x);
}
```

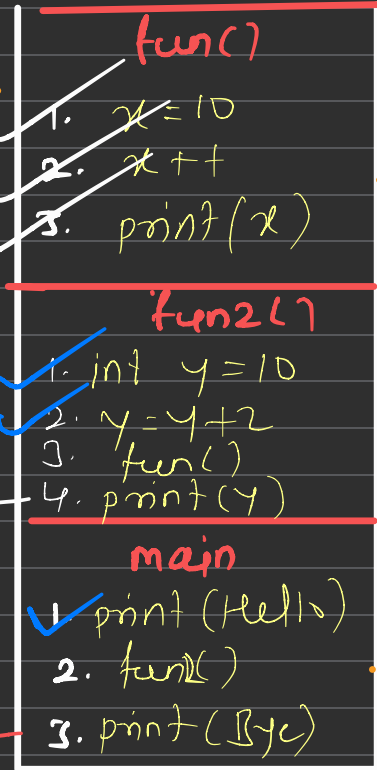
```
void fun2()
{
    int y = 10;
    y = y + 2;
    fun1();
    print(y);
}
```

```
main()
{
    print("Hello");
    fun2();
    print("Bye");
}
```

Hello 11 12 Bye

waiting

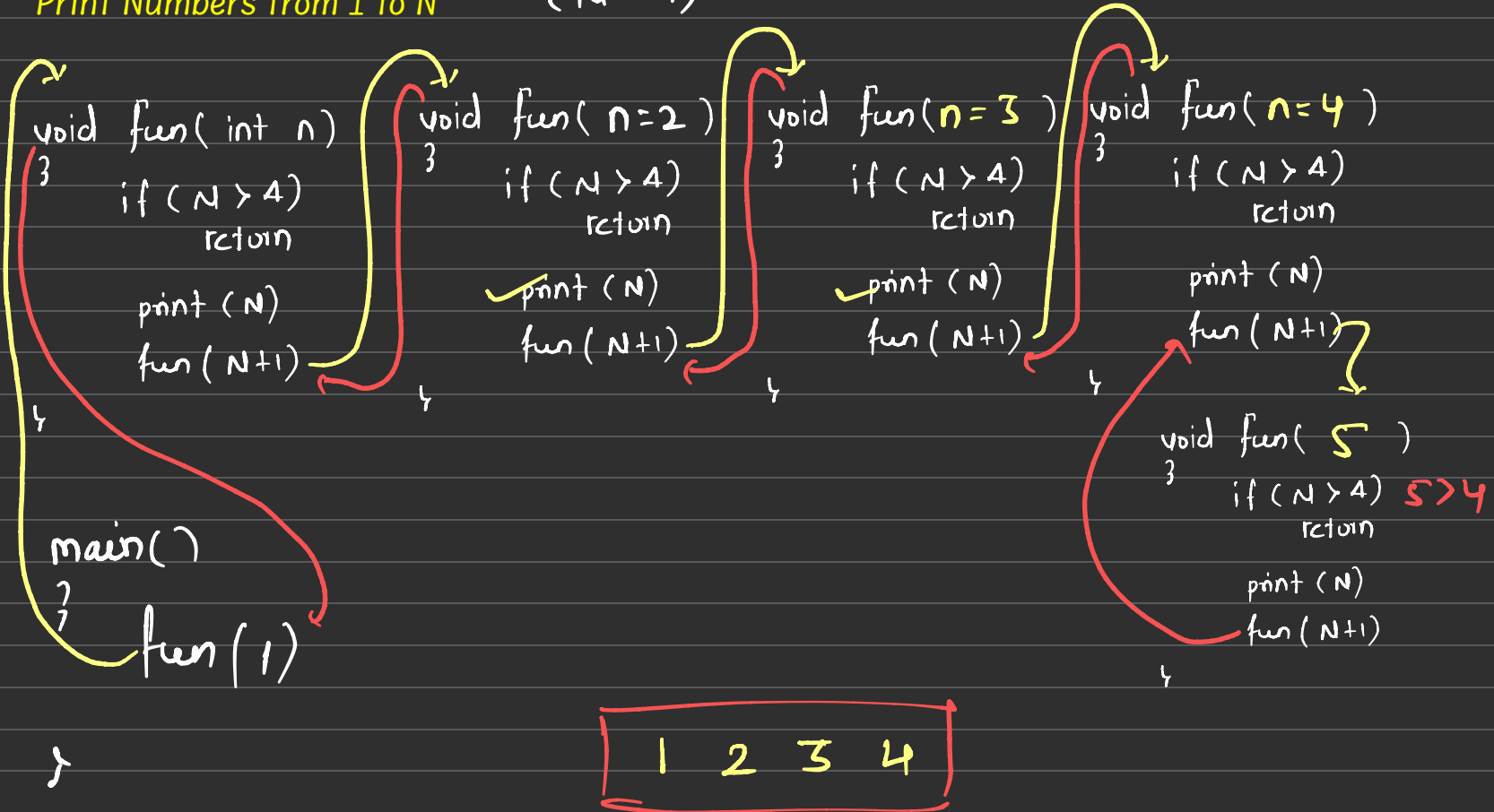
waiting



stack

Print Numbers from 1 to N

(N = 4)



Print Numbers from N to 1 (N=4)

```
void fun(n=4)
```

```
{  
  if (n ≤ 0)  
    return
```

```
  print(n)  
  fun(N-1)  
}
```

```
void fun(n=3)
```

```
{  
  if (n ≤ 0)  
    return
```

```
  print(n)  
  fun(N-1)  
}
```

```
void fun(int n)
```

```
{  
  if (n ≤ 0)  
    return
```

```
  print(n)  
  fun(N-1)  
}
```

```
void fun(int n)
```

```
{  
  if (n ≤ 0)  
    return
```

```
  print(n)  
  fun(N-1)  
}
```

```
void fun(int n)
```

```
{  
  if (n ≤ 0)  
    return
```

```
  print(n)  
  fun(N-1)  
}
```

```
main()
```

```
{  
  fun(N)  
}
```

4 3 2 1

N=4

print N to 1 But start from 1

```
void fun(1)
{
  if (N == 4)
    return
```

```
  fun(N+1)
  print(N)
}
```

```
void fun(2)
{
  if (N == 4)
    return
```

```
  fun(N+1)
  print(N)
}
```

```
void fun(3)
{
  if (N == 4)
    return
```

```
  fun(N+1)
  print(N)
}
```

```
void fun(4)
{
  if (N == 4)
    return
```

```
  fun(N+1)
  print(N)
}
```

```
main()
{
  fun(1)
}
```

3 2 1

Before

print (N)

fun (N+1)



print in same
order

After

fun (N+1)

print (N)

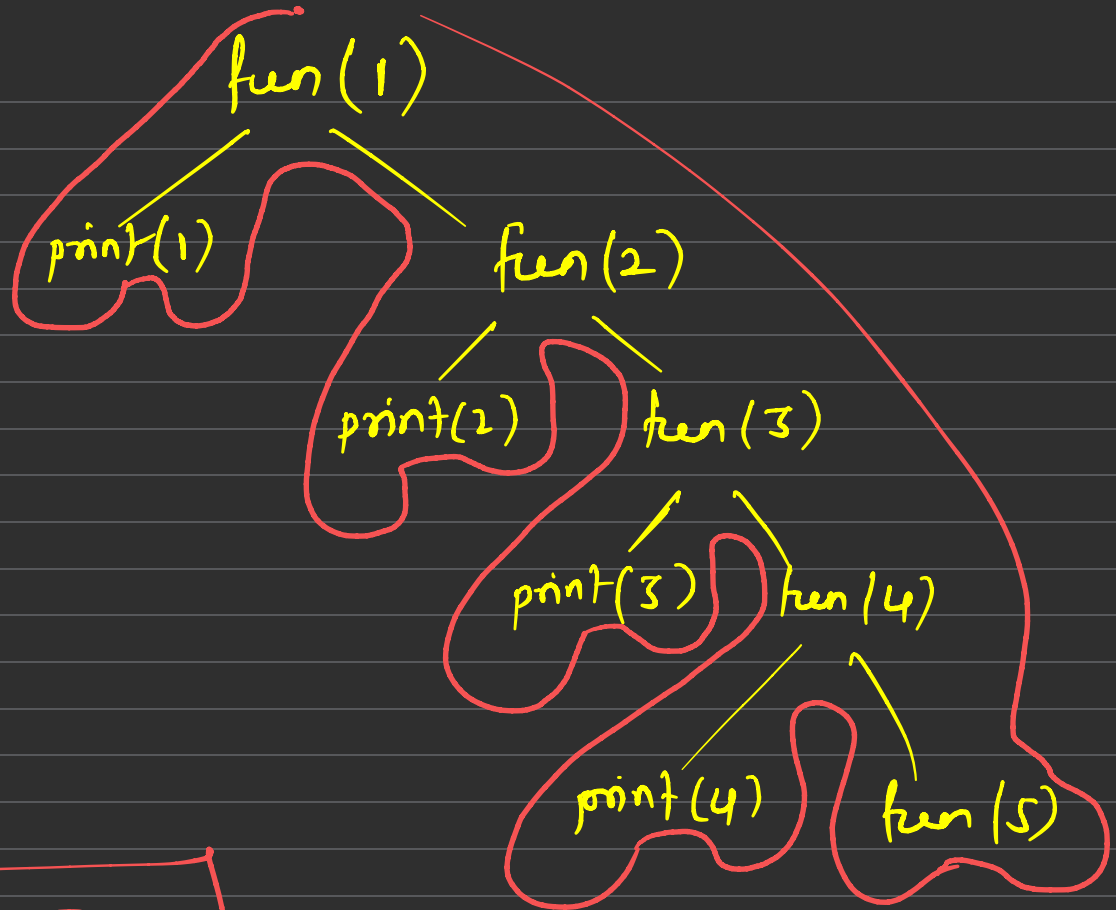


print in
Reverse order

Recursive Tree

```
void fun(N)
3  if (N > 4)
    return
    ✓ print(N)
    ✓ fun(N+1)
```

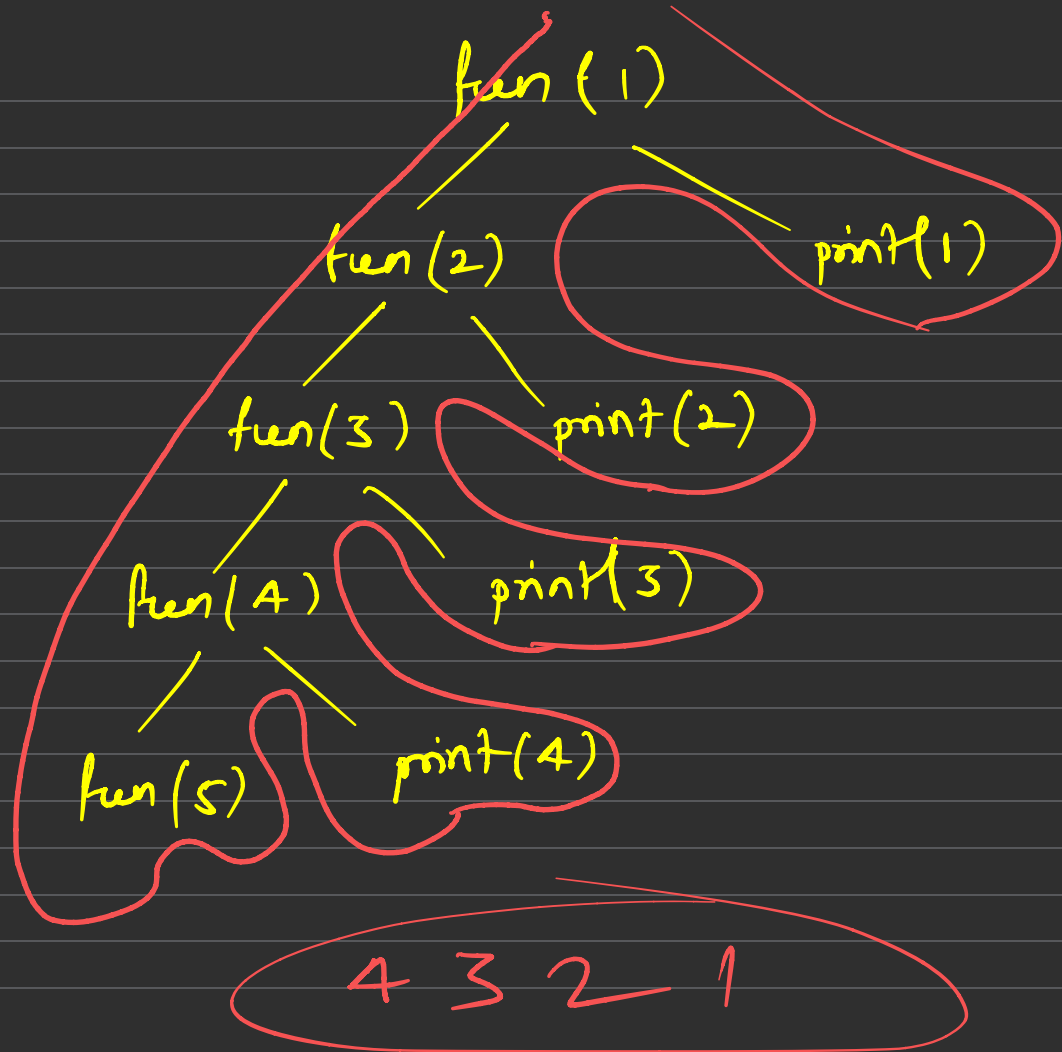
~

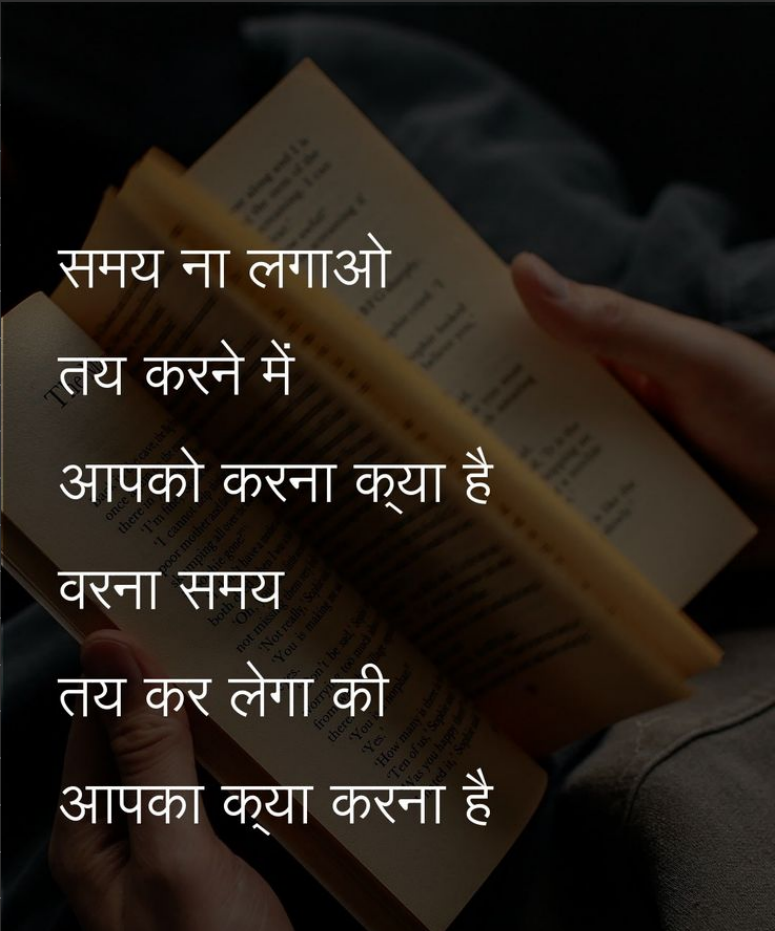


1 2 3 4

```
void fun(N)
{
    if (N == 5)
        return;

    fun(N+1);
    print(N);
}
```



A close-up photograph of a person's hands holding an open book. The book's pages are yellowed with age and contain printed text. The person is wearing a grey long-sleeved shirt. The background is dark and out of focus. Overlaid on the image is a white text in Hindi.

समय ना लगाओ
तय करने में
आपको करना क्या है
वरना समय
तय कर लेगा की
आपका क्या करना है

Parameterised and Functional Recursion

Parameterised Recursion

Find the Sum of all the numbers from 1 to N

Find the Sum of all the numbers from 1 to N

Sum = ~~0~~ ~~1~~ ~~2~~ ~~3~~ ~~4~~ 10

^{1 4 0}
void fun (i , N , sum)
{
 if (i > N)
 {
 print (sum)
 return
 }
 sum = sum + i
 fun (i+1 , N , sum)
}

void fun (2 , 4 , 1)
{
 if (i > N)
 {
 print (sum)
 return
 }
 sum = sum + i
 fun (i+1 , N , sum)
}

void fun (3 , 4 , 3)
{
 if (i > N)
 {
 print (sum)
 return
 }
 sum = sum + i
 fun (i+1 , N , sum)
}

void fun (4 , 4 , 6)
{
 if (i > N)
 {
 print (sum)
 return
 }
 sum = sum + i
 fun (i+1 , N , sum)
}

void fun (5 , 4 , 10)
{
 if (i > N)
 {
 print (sum)
 return
 }
 sum = sum + i
 fun (i+1 , N , sum)
}

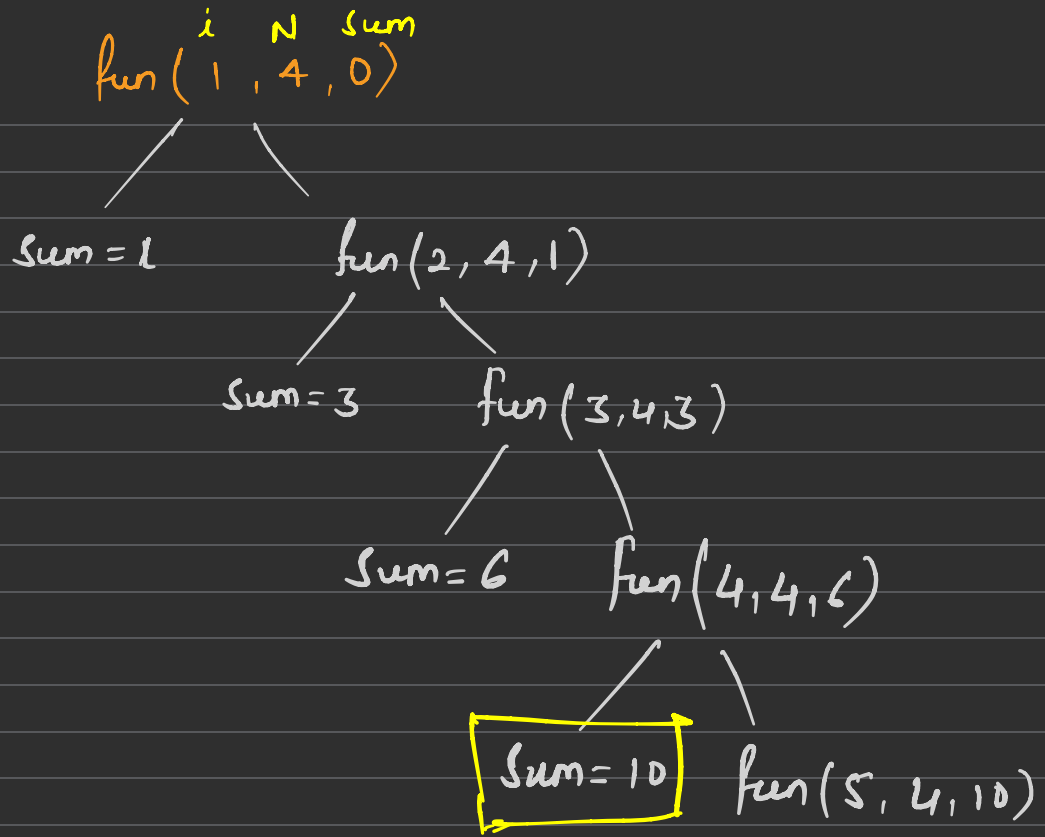
void main ()
{
 N = 4
 sum = 0

fun (1 , N , sum) print (sum) = 10

```

void fun ( i , N , sum)
{
    if ( i > N)
    {
        print (sum)
        return
    }
    sum = sum + i
    fun ( i+1 , N , sum)
}

```



Functional Recursion

Find the Sum of all the numbers from 1 to N

^{1, 4}
int fun (i, N)
{
if (i > N)
return 0
1 +
return i + fun (i+1, N) }

^{2, 4}
int fun (i, N)
{
if (i > N)
return 0
2 +
return i + fun (i+1, N) }

^{3, 4}
int fun (i, N)
{
if (i > N)
return 0
3 +
return i + fun (i+1, N) }

^{4, 4}
int fun (i, N)
{
if (i > N)
return 0
4 +
return i + fun (i+1, N) }

^{5, 4}
int fun (i, N)
{
if (i > N)
return 0
return i + fun (i+1, N) }

void main()
{
N = 4

int ans = ~~fun(1, N)~~ 10

cout << ans

10

```

int fun ( i, N)
{
    if ( i > N)
        return 0

    return i + fun ( i+1, N)
}

```

$$\text{fun}(1, 4) = 10$$

$$1 + \text{fun}(2, 4) = 9$$

$$2 + \text{fun}(3, 4) = 7$$

$$3 + \text{fun}(4, 4) = 4$$

$$4 + \text{fun}(5, 4) = 0$$

HW problems

Addition of digits in Number → Functional + parameter

Write a code to find the factorial of a number → Functⁿ + parameter

Program to count digits in a number

Program to check if string is palindrome or not

Reverse array Using Recursion

```
void reverse(a[], i, j)
{
    if (i >= j)
        return;

    swap(a[i], a[j]);
    reverse(a, i+1, j-1);
}
```

```
void main()
{
    reverse(arr, 0, N-1);
}
```

1	2	3	4	5	6	7
0	1	2	3	4	5	6

$i = 0, j = N - 1$

```
while (i < j)
{
    swap(a[i], a[j]);
    i++;
    j--;
}
```


0 6
 void reverse(a[], i, j)
 {
 if (i >= j)
 return

✓ swap(a[i], a[j])
 reverse(a, i+1, j-1)

}

1 5
 void reverse(a[], i, j)
 {
 if (i >= j)
 return

✓ swap(a[i], a[j])
 reverse(a, i+1, j-1)

}

2 4
 void reverse(a[], i, j)
 {
 if (i >= j)
 return

✓ swap(a[i], a[j])
 reverse(a, i+1, j-1)

}

3 3
 void reverse(a[], i, j)
 {
 if (i >= j)
 return

swap(a[i], a[j])
 reverse(a, i+1, j-1)

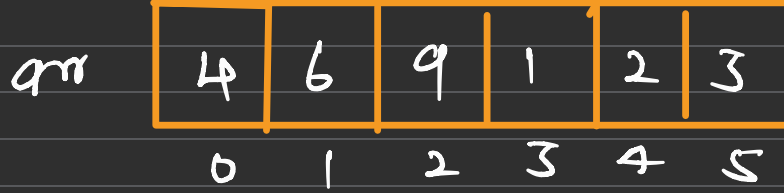
}

7	6	5	3	2	1	
1	2	3	4	5	6	7
0	1	2	3	4	5	6

size of int
4 Byte

$N = 6$

$N = 6$
Total size = 24



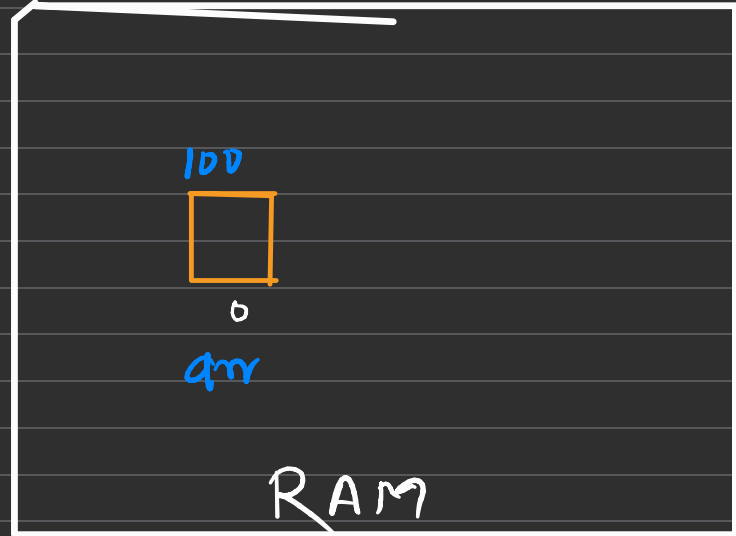
arr[3]

*(arr + 3)

*(100 + 3) = (100 + 12)

*(112)

||



int arr[5]

	100	104	108	112	116
arr	1	6	'a'	q	'z'
	0	1	2	3	4

* (arr + 3)

Int = 4B

char = 1B

* (100 + 3 * 4) = * (100 + 12)

(112)

new
memory
calc

